B2B Distributor Ordering Platform

Product Requirements and Discovery Baseline

Prepared from the approved project documentation and verified POS architecture findings

Status: Discovery baseline	**Version:** 1.0

October 2026

1. Status and objective

This document establishes the discovery baseline for a retailer-facing B2B wholesale ordering platform that will integrate with the existing POS while preserving the POS as the primary source of truth. Product-specific details remain subject to project-owner approval.

Objective

Create a secure, scalable B2B ordering experience for retailers while reusing verified POS business logic, avoiding unnecessary data duplication, preserving existing operations, and leaving room for future multi-distributor support.

2. Current POS baseline

The imported POS is a TanStack Start application using React, TypeScript, Vite, TanStack Router, TanStack Query, and Supabase/PostgreSQL. It currently provides modules for billing, products, inventory, customers, suppliers, purchases, invoices, returns, expenses, reports, settings, and dashboard operations.

The POS database includes products, categories, batches, stock movements, customers, bills, bill items, payments, purchase orders, suppliers, user roles, profiles, audit logs, day closes, and application settings.

Its **create_bill** database function performs bill creation, batch stock deduction, stock-movement recording, payment recording, customer-balance updates, audit logging, and MRP-ceiling validation. These are discovery findings, not permission to couple the storefront directly to POS tables.

3. Planned platform areas

Existing POS

- ? Products and categories
- ? Inventory and batch tracking
- ? Billing and invoices
- ? Customers and receivables
- ? Pricing and wholesale prices
- ? Purchases, suppliers, expenses, reports, and business operations

B2B storefront

- ? Retailer login and account access
- ? Product browsing and category navigation
- ? Wholesale pricing and quantity-based discounts
- ? Shopping cart and checkout
- ? Order submission and order history
- ? Order tracking and status visibility

Integration layer

- ? Product synchronization
- ? Inventory synchronization
- ? Price synchronization
- ? Retailer synchronization

? Online-order synchronization

? Order-status synchronization

4. Data ownership

Domain	Initial owner	Storefront responsibility
Products and categories	Existing POS	Consume an approved catalog contract or projection
Inventory availability	Existing POS	Display availability subject to synchronization guarantees
Wholesale prices and discounts	POS/business rules	Display only approved prices and discount rules
Retailer identity	To be confirmed	Authenticate and manage retailer-facing profile
Online orders	Integration boundary with POS acceptance	Submit idempotently and show synchronization status
Invoices, payments, operations	Existing POS	Display approved order/invoice status only

5. Non-functional requirements

? Preserve existing POS functionality.

? Maintain secure authentication and authorization.

? Avoid unnecessary duplication of product and inventory data.

? Keep integration concerns separate from storefront concerns.

? Support future multi-distributor architecture.

? Provide auditable synchronization and order state transitions.

? Preserve historical price and order-line snapshots.

? Make synchronization retries, failures, and reconciliation observable.

? Support tenant and distributor isolation in future deployments.

6. Open questions

? What approved API or service boundary should expose POS-owned data?

? Which POS entities and business rules are safe and supported for reuse?

? What are the authoritative identifiers for products, retailers, prices, inventory, and orders?

? What synchronization frequency and consistency guarantees are required?

? Which distributor-specific rules must be configurable?

? What retailer roles, permissions, and onboarding flows are required?

? Should inventory be real-time, periodically projected, or reservation-backed?

? How should unavailable, changed, cancelled, or partially fulfilled orders be reconciled?

7. Implementation-readiness criteria

Implementation must not begin until all of the following are satisfied:

- ? The POS architecture and supported extension points are documented.
- ? Ownership and synchronization direction are approved for every shared domain.
- ? Retailer roles, authorization rules, and onboarding are defined.
- ? Product, price, inventory, and order identifiers are agreed.
- ? Failure, retry, reconciliation, audit, and observability behavior is specified.
- ? A technology and deployment decision is recorded in planning/DECISIONS.md.

8. Governing architecture rules

- ? The existing POS remains the primary source of truth.
- ? Reuse existing POS business logic wherever practical.
- ? Do not create duplicate product or inventory databases unnecessarily.
- ? Keep integration code separate from the storefront.
- ? Do not select or install a new technology stack until POS inspection is complete.
- ? Do not invent POS functionality, APIs, schemas, or business rules.
- ? Use explicit contracts and least-privilege access for future integration.

9. Source documentation

This PDF is generated from the maintained Markdown source: **documentation/product-requirements/B2B-Platform-Requirements.md**. Supporting discovery records are maintained in **documentation/architecture/SYSTEM_ARCHITECTURE.md** and **documentation/database/DATABASE_DESIGN.md**.